Lab 3

4.2 Background Exercises

Submit these background exercises with the lab report.

4.2.1 Synthesis of Periodic Signals

Each signal given below represents one period of a periodic signal with period T_0.

For each of these two signals, do the following:

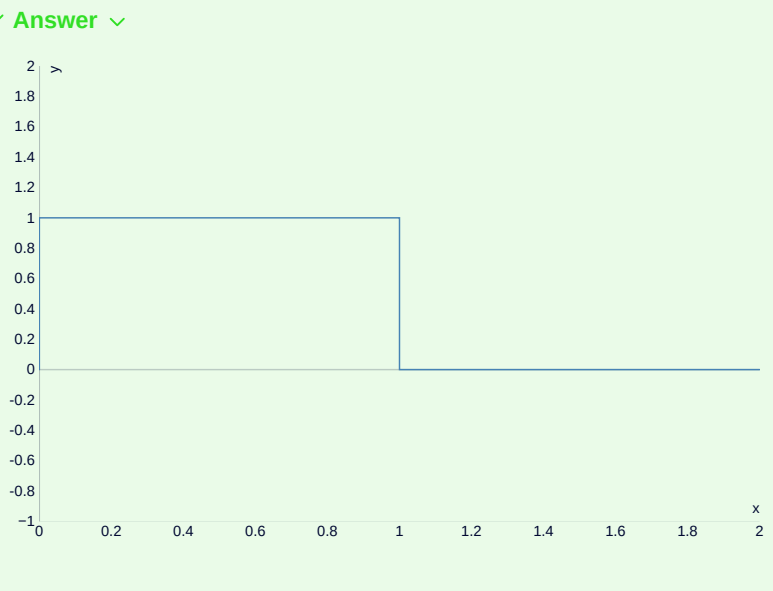
- Sketch the signal on the interval $[0, T_0]$.
- Compute the Fourier series expansion in the form

$$s(t) = a_0 + \sum_{k=1}^{\infty} A_k \sin(2\pi k f_0 t + \theta_k)$$

1

Period $T_0 = 2$, For $t \in [0, 2] : s(t) = \text{rect}\left(t - \frac{1}{2}\right)$

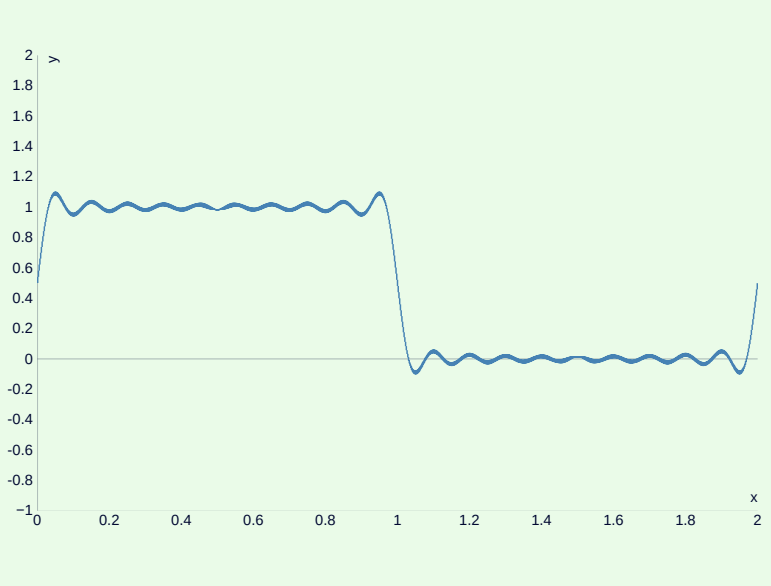
✓ Answer



$f_0 = \frac{1}{T_0} = \frac{1}{2}$

Since this function is odd, all terms will be sine terms.

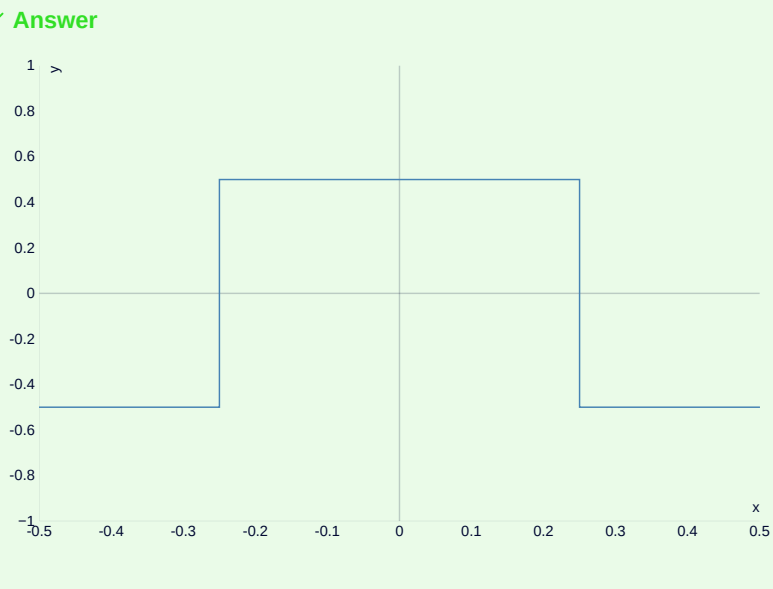
$a_0 = \frac{1}{2}$
$a_k = 0$
$b_k = 2f_0 \int_0^2 s(t)\sin(2\pi kt)\,dt = 2f_0 \int_0^1 \sin(2\pi kt)\,dt = \frac{1}{\pi k}(1 - (-1)^k)$
$s(t) = \frac{1}{2} + \sum_{k=1}^{\infty} \frac{1}{\pi k}(1 - (-1)^k)\sin(\pi kt)$



2

Period $T_0 = 1$, For $t \in \left[-\frac{1}{2}, \frac{1}{2}\right] : s(t) = \text{rect}(2t) - \frac{1}{2}$

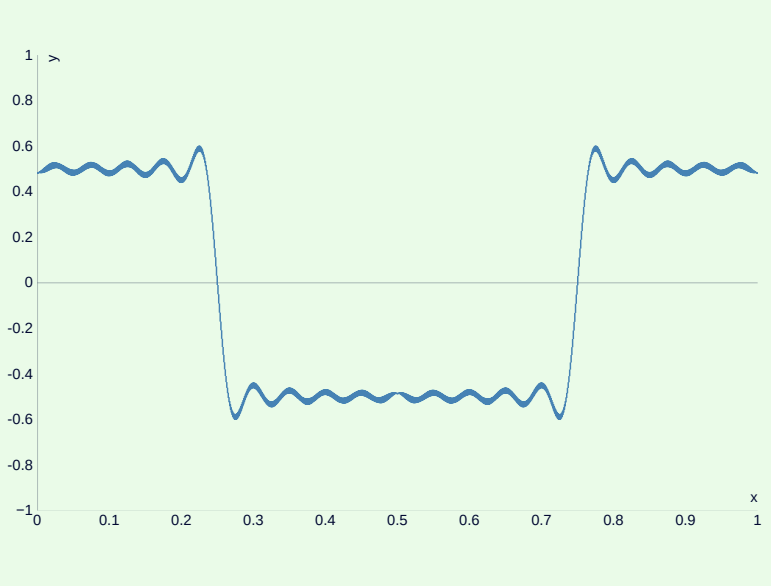
✓ Answer



$f_0 = \frac{1}{T_0} = 1$

Since this function is even, all the terms will be cosine terms.

$a_0 = 0$
$b_k = 0$
$a_k = 2f_0 \int_{-1/2}^{1/2} s(t)\cos(2\pi kt)\,dt = 2f_0 \int_{-1/4}^{1/4} s(t)\cos(2\pi kt)\,dt = \frac{2}{\pi k}\sin(\frac{\pi k}{2})$
$s(t) = \sum_{k=1}^{\infty} \frac{2}{\pi k}\sin(\frac{\pi k}{2})\cos(2\pi kt)$
$= \sum_{k=1}^{\infty} \frac{2}{\pi k}\sin(\frac{\pi k}{2})\sin(2\pi kt + \frac{\pi}{2})$



4.2.2 Magnitude and Phase of Discrete-Time Systems

For the discrete-time system described by the following difference equation,

$$y(n) = 0.9y(n-1) + 0.3x(n) + 0.24x(n-1)$$

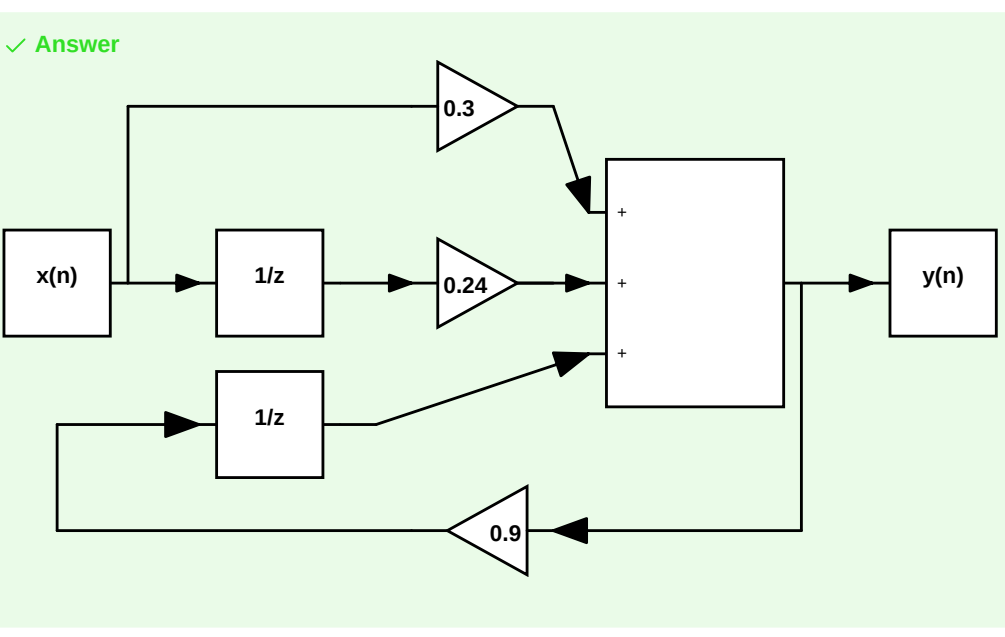
Compute the impulse response.

✓ Answer

$x(t) = \delta(t)$
$y(-1) = 0$
$h(0) = 0.3$
$h(1) = 0.9(0.3) + 0.24 = 0.51$
$h(n > 1) = 0.9y(n-1)$
$h(n) = \begin{cases} 0.3 & n = 0 \\ 0.51(0.9)^{n-1} & n \geq 1 \end{cases}$

Draw a system diagram.

✓ Answer



Take the Z-transform of the difference equation using the linearity and the time shifting properties of the Z-transform.

✓ Answer

$y(n) \rightarrow Y(z)$
$0.9y(n-1) \rightarrow 0.9z^{-1}Y(z)$
$0.3x(n) \rightarrow 0.3X(z)$
$0.24x(n-1) \rightarrow 0.24z^{-1}X(z)$
$Y(z) = 0.9z^{-1}Y(z) + 0.3X(z) + 0.24z^{-1}X(z)$

Find the transfer function, defined as

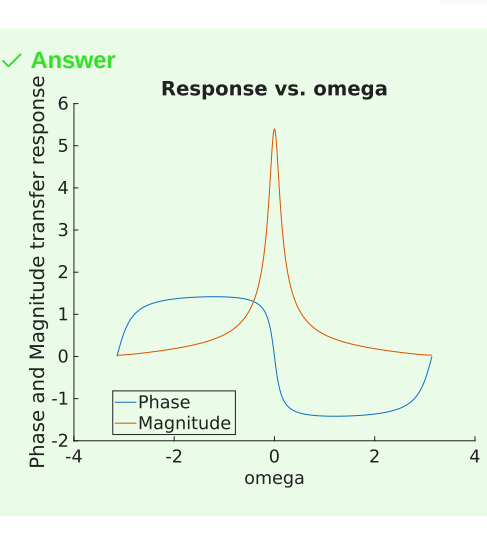
$$H(z) = \frac{Y(z)}{X(z)}$$

✓ Answer

$Y(z) = 0.9z^{-1}Y(z) + 0.3X(z) + 0.24z^{-1}X(z)$
$Y(z)(1 - 0.9z^{-1}) = X(z)(0.3 + 0.24z^{-1})$
$\frac{Y(z)}{X(z)} = \frac{0.3+0.24z^{-1}}{1-0.9z^{-1}}$
$H(z) = \frac{0.3z+0.24}{z-0.9}$

Use Matlab to compute and plot the magnitude and phase responses, $|H(e^{j\omega})|$ and $\angle H(e^{j\omega})$, for $-\pi < \omega < \pi$. You may use Matlab commands `phase` and `abs`.

✓ Answer

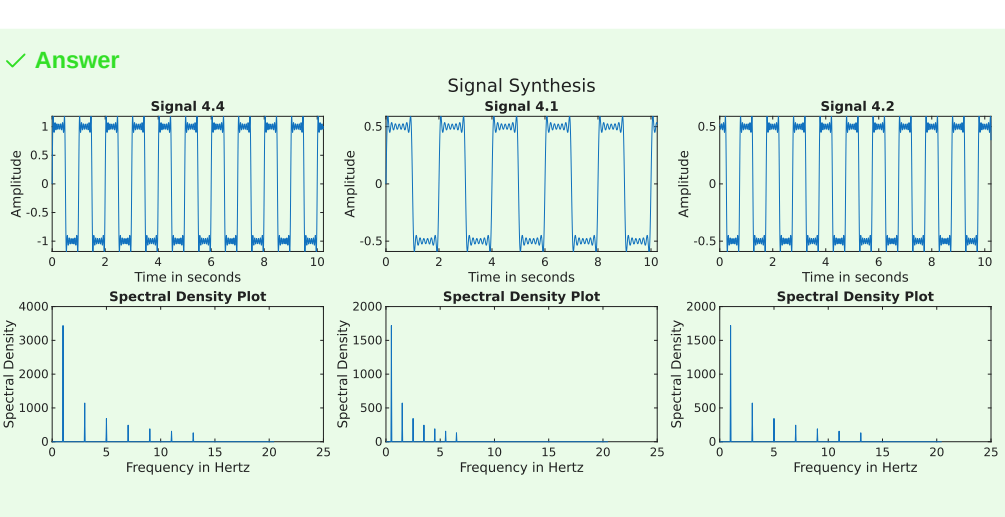


4.4 Continuous-Time Frequency Analysis

4.4.1 Synthesis of Periodic Signals

Hand in plots of the Spectrum Analyzer output for each of the three synthesized waveforms. For each case, comment on how the synthesized waveform differs from the desired signal, and on the structure of the spectral density.

✓ Answer



The synthesized waveform appears to not handle sharp changes well, oscillating a larger amount around sharp change points as compared to other points among the waveform.

The spectral density appears to match the composite signals very well, with a peak at each frequency and amplitudes correlating with that of the decomposed signals. It also appears that at higher frequencies, the intensities diminish as the original signal did not have high frequency information.

4.4.2 Modulation Property

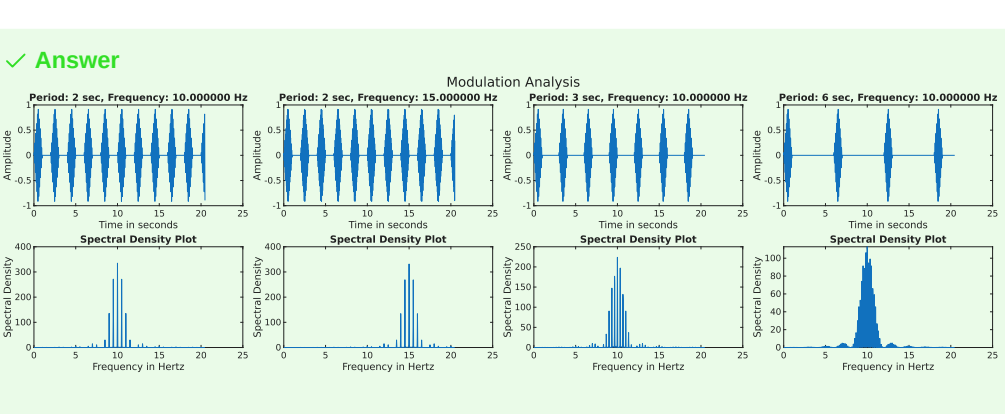
Hand in plots of the output of the Spectrum Analyzer for each signal.

What effect does changing the modulating frequency have on the spectral density?

Why does the spectrum have a comb structure and what is the spectral distance between impulses? Why?

What would happen to the spectral density if the period of the triangle pulse were to increase toward infinity? (in the limit)

✓ Answer



Changing the modulating frequency shifts the larger peak over to that of the modulating frequency. For each of the 10Hz modulations, the peak of the comb is at 10Hz, and similarly for that of the 15Hz modulation.

The comb structure corresponds to the transmission of the modulated frequency, with each smaller peak being spread out by the frequency of the modulated signal itself. This is due to the fact that multiplied signals have a convolution of their frequencies, leading to the combed shape.

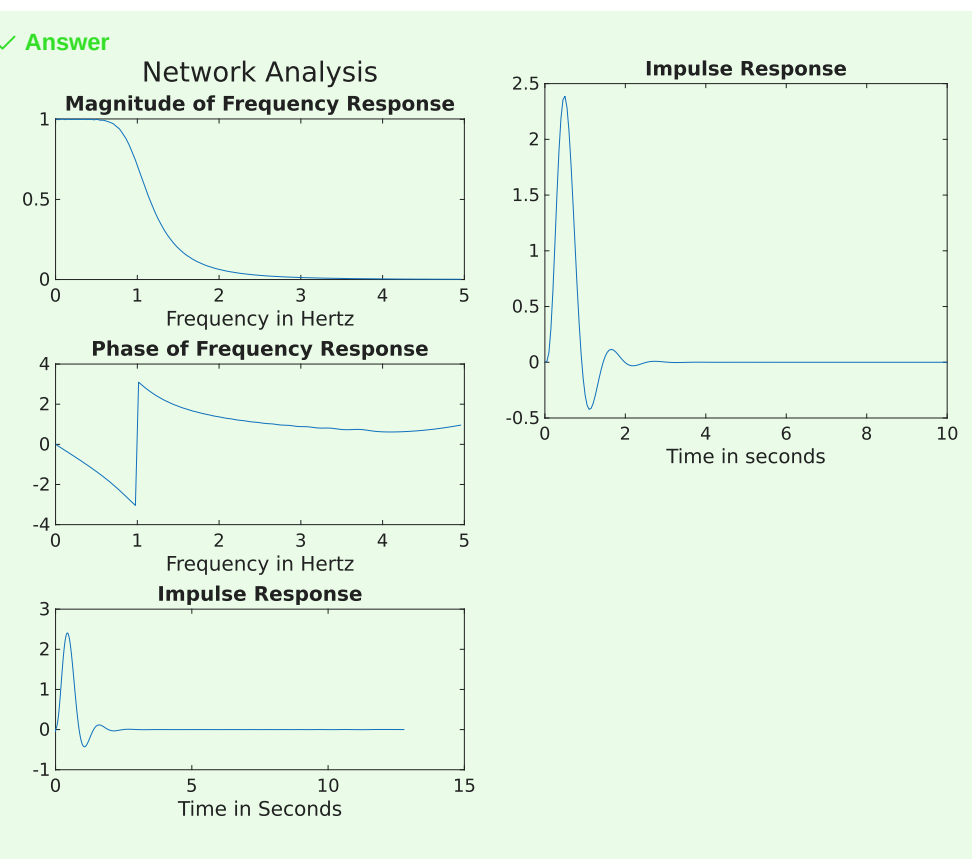
The density of the spectral density would increase as when the period of the pulse wave increases, its frequency decreases, making the comb structure tighter. As the period approaches infinity, the comb would be infinitely dense. The increase in density of the spectral density would in turn reduce the maximum amplitude of spectral density.

4.4.3 System Analysis

Hand in the printout of the output of the Network Analyzer (magnitude and phase of the frequency response, and the impulse response) and the plot of the impulse response obtained using a unit step.

What are the advantages and disadvantages of each method?

✓ Answer



The Network Analysis gives more than just the input response, it also provides direct frequency response, making it easier to do frequency analysis directly. It does require the synthesis of a chirp signal as well as recording the feedback from the DUT.

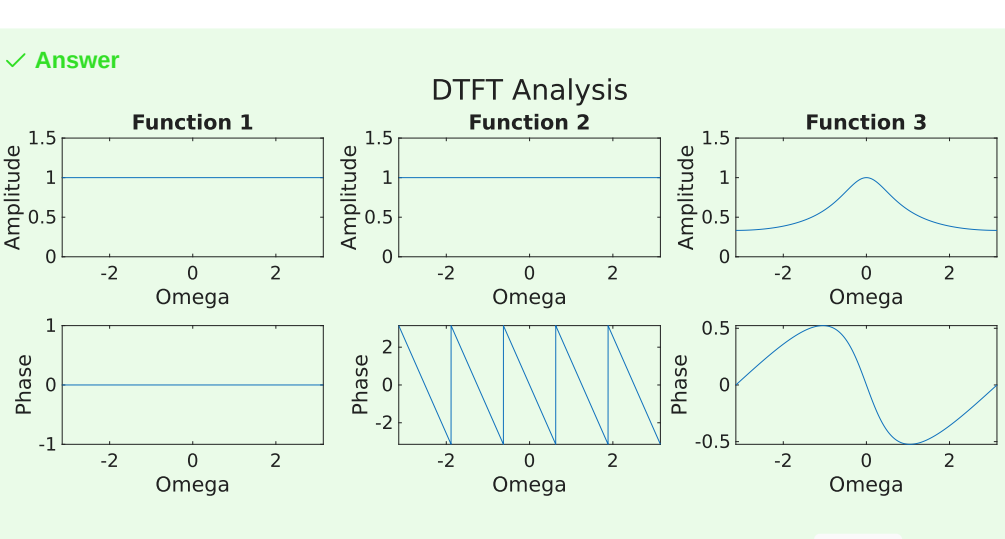
Alternatively, the step has a simpler setup, not requiring synthesizing a chirp, nor feedback from the DUT. It does require you to take a derivative however, which may be difficult to implement in hardware. It also does not provide the frequency analysis of the DUT directly.

4.5 Discrete-Time Frequency Analysis

4.5.1 Discrete-Time Fourier Transform

Hand in a printout of your Matlab function. Also hand in plots of the DTFT's magnitude and phase for each of the three signals.

✓ Answer



The matlab function may be found in the Appendix of this document under file `q451.m`.

4.5.2 System Analysis

Hand in:

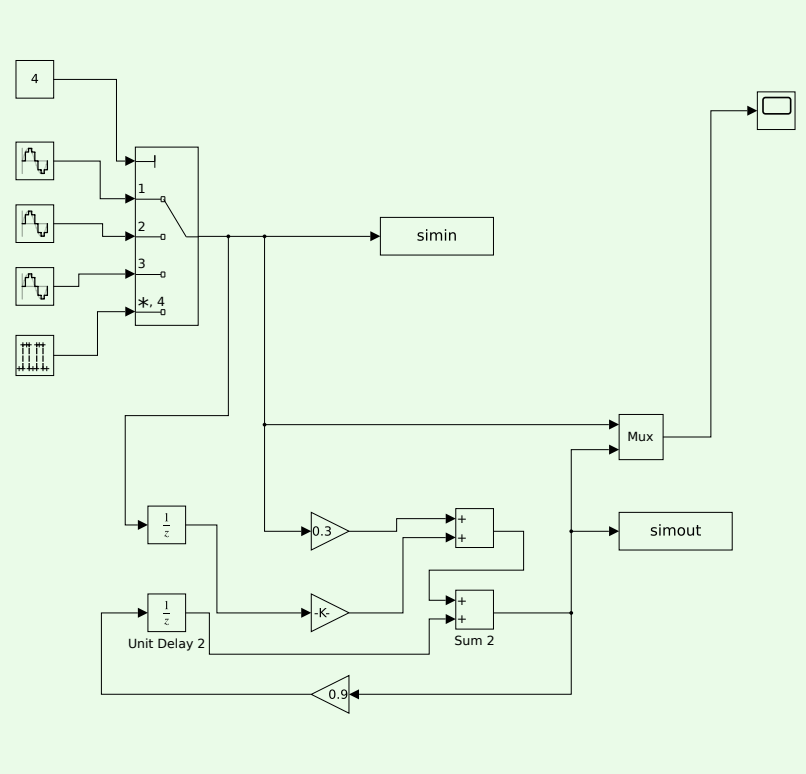
Printout of your completed block diagram.

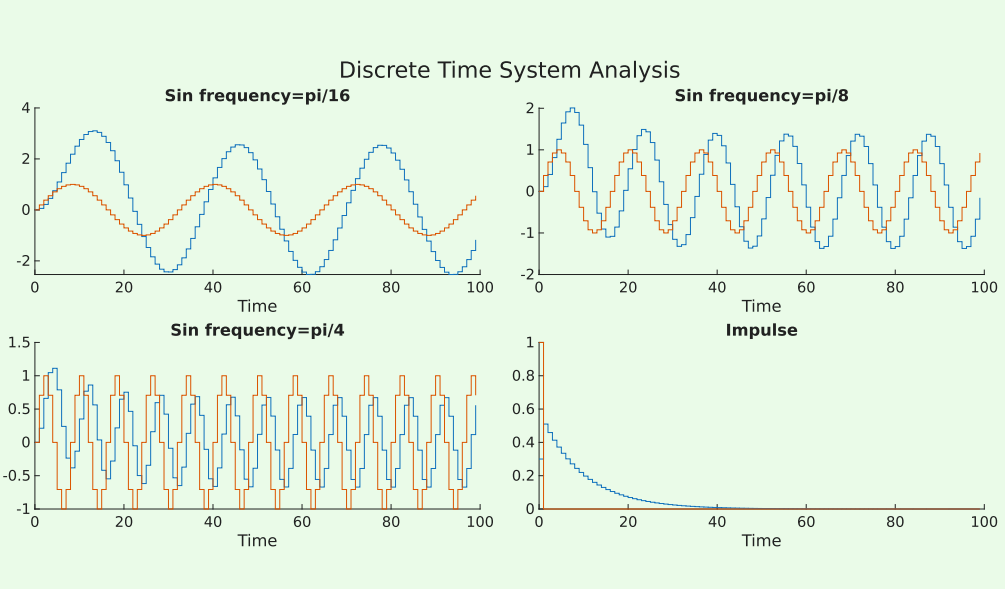
Table of both the amplitude measurements you made and their theoretical values.

Printout of the figure with the impulse response, and the magnitude and phase of the frequency response.

✓ Answer

Completed Block Diagram:





Plotted are the 3 different sine parameters given, along with the impulse response, where Orange is the input signal, and blue is our output signal.

Frequency	Experimental	Theoretical ($\|H(e^{j\omega})\|$)	Error
$\pi/16$	2.535499	2.5451993170216354	-0.3811%
$\pi/8$	1.372979	1.3816250517356978	-0.6258%
$\pi/4$	0.671294	0.6813926185794887	-1.4821%

Code

`helpers.m`

```
% A function to help set our defaults for all plots
function [fontSize, export] = helpers(f, varargin)
    p = inputParser;

    % Define defaults
    addParameter(p, 'w', 1000);
    addParameter(p, 'h', 800);
    parse(p, varargin{:});

    % Disables LaTeX rendering, as it is not supported by the export function well
    set(f,'defaultTextInterpreter','none')
    set(f,'defaultAxesTickLabelInterpreter','none')
    set(f,'defaultLegendInterpreter','none')
    % Increase Default font size
    defaultFontSize = 25;
    set(f, 'defaultTextFontSize', defaultFontSize);
    set(f, 'defaultAxesFontSize', defaultFontSize);
    set(f, 'defaultLegendFontSize', defaultFontSize);

    % Set Font
    set(f, 'DefaultAxesFontName', 'Arial');
    set(f, 'DefaultTextFontName', 'Arial');
    set(f, 'DefaultLegendFontName', 'Arial');
    set(f, 'DefaultColorbarFontName', 'Arial');

    % Set clear background
    set(f, 'DefaultFigureColor', 'none');
    set(f, 'DefaultAxesColor', 'none');
    set(f, 'DefaultLegendColor', 'none');
    set(f, 'DefaultFigureInvertHardcopy', 'off');

    % Set the default layout to "loose"
    set(f, 'DefaultTiledLayoutTileSpacing', 'loose');
    set(f, 'DefaultTiledLayoutPadding', 'loose');

    % Increase Canvas size
    set(f, 'Position', [0 0 p.Results.w p.Results.h]);

    % Set text alignment properties
    set(f, 'DefaultAxesTitleHorizontalAlignment', 'center');
    set(f, 'DefaultTextHorizontalAlignment', 'center');

    % Disable slow output warning
    warning('off', 'MATLAB:print:ContentTypeImageSuggested')

    % Set high quality output by default
    set(f, 'DefaultFigureRenderer', 'painters');
    f.Visible = "on";
    % Return our font size
    fontSize = defaultFontSize;
    export = @(fp) exportgraphics(f, fp, ContentType='vector', BackgroundColor='none');
end
```

`plotNet.m`

```
% Plot Network Analysis given axes, time, input samples, output samples, and number of samples to plot
function plotNet(pl, t, ins, outs, N)
    % Find total samples and Ts
    Nn = numel(ins);
    Ts = (t(Nn) - t(1))/Nn;

    % Calculate FFTs of inputs and outputs
    outsf = fft(outs);
    insf = fft(ins);

    % Calculate our transfer function
    transfer = outsf./insf;
    Fs = 1/Ts;
    % Resample our x-axis for frequency
    f = (0:N-1) * (Fs/Nn);

    % Plot Magnitude and Phase of the response
    pls = nexttile(pl);
    plot(pls, f(1:(N/2)), abs( transfer(1:(N/2))) );
    xlabel(pls, 'Frequency in Hertz');
    title(pls, 'Magnitude of Frequency Response');

    pls = nexttile(pl);
    plot(pls, f(1:(N/2)), angle( transfer(1:(N/2))) );
    xlabel(pls, 'Frequency in Hertz');
    title(pls, 'Phase of Frequency Response');

    % Plot our Impulse response using ifft
    pls = nexttile(pl);
    h = ifft(transfer);
    h=h/Ts;
    plot(pls, t(1:(round(Nn/2))), real(h(1:(round(Nn/2)))) );
    xlabel(pls, 'Time in Seconds');
    title(pls, 'Impulse Response');
end
```

`plotSpec.m`

```
% Plot Spectran Analysis given axes, time, signal, number of items to plot, and title
function plotSpec(pl, t, signal, N, ti)
    % Find total samples and Ts
    Nn = numel(signal);
    Ts = (t(Nn) - t(1))/Nn;

    % Plot Signal itself
    pls = nexttile(pl);
    plot(pls, t(1:2*N), signal(1:2*N));
    xlabel(pls, 'Time in seconds');
    ylabel(pls, 'Amplitude');
    title(pls, ti);

    % Calculate the amplitude of the FFT
    X = abs(fft( hamming(Nn).*signal ));
    f = (1/Ts)*((1:Nn)-1)./Nn;

    % Plot FFT analysis
    pls = nexttile(pl);
    plot(pls, f(1:N),X(1:N));
    xlabel(pls, 'Frequency in Hertz');
    ylabel(pls, 'Spectral Density');
    title(pls, 'Spectral Density Plot');
end
```

`q422.m`

```
% Define our systems as defined in the lab
func = @(z) (0.3.*z+0.24)./(z-0.9);
pfunc = @(o) func(exp(1i*o));

% Define our domain and sample our function (delta)
domain = linspace(-pi, pi, 1000);
resp = pfunc(domain);

% Set up canvas
f = figure;
[ff, e] = helpers(f);

% Plot angle and amplitude
hold on
plot(domain, angle(resp))
plot(domain, abs(resp))
legend( ...
    'Phase', ...
    'Amplitude', ...
    Location='best', ...
    FontSize=ff )
xlabel("omega", FontSize=ff)
ylabel("Phase and Radius transfer response")
title("Response vs. omega")

% Export graphic
e('plots/1_q422.svg')
```

`q411.m`

```
% Add the simulink files to path
addpath("simu")

% The model we are working on
mdl = 'lab3Synth';

% Load simulink model and turn on logging
system = load_system(mdl);
set_param(mdl, ReturnWorkspaceOutputs="on")

% Define which Sine Blocks we are modifying
blocks = 1:7;

% Parameter names to alter
param_names = {"Amplitude", "Frequency", "Phase"};

% Define parameters for the Sine Blocks for each signal
amplitudes = { (@(i) 4/(pi*(2*i-1))), (@(i) 2/(pi*(2*i-1))), (@(i) 2*(-1)^(i-1)/(pi*(2*i-1))) };
frequencies = { (@(i) 2*pi*(2*i-1)), (@(i) pi*(2*i-1)), (@(i) 2*pi*(2*i-1)) };
phases = { (@(i) 0), (@(i) 0), (@(i) pi/2)};
titles = { "Signal 4.4", "Signal 4.1", " Signal 4.2" };

params = [amplitudes; frequencies; phases];

% Our output plot length
N = 1024;

% Plot each spectral analysis
f = figure;
[ff, e] = helpers(f, w=3000, h=1200);
t = tiledlayout(2, 3, TileIndexing="columnmajor");

for i=1:size(params, 1)
    for j=1:numel(param_names)
        % Set relevant parameters
        for k=blocks
            set_param(sprintf("%s/Sine Wave%d", mdl, k), param_names{j}, sprintf("%f", params{j, i}(k)));
        end
    end
    % Run simulation
    outs = sim(mdl);
    % Plot spectral analysis on our figure
    plotSpec(t, outs.simout.time, outs.simout.data, N, titles{i});
end

% Title toplevel
title(t, "Signal Synthesis", FontSize=ff*1.5);
% Export full graph
e("plots/2_q441.svg")

% Clean up
close_system(mdl, 0);
Simulink.sdi.clear;
```

`q442.m`

```
% Add the simulink files to path
addpath("simu")

% The model we are working on
mdl = 'lab3Mod';

% Load simulink model and turn on logging
system = load_system(mdl);
set_param(mdl, ReturnWorkspaceOutputs="on")

periods = [2, 2, 3, 6];
freqs = [10, 15, 10, 10];

% Our output plot length
N = 1024;

% Plot each spectral analysis
f = figure;
[ff, e] = helpers(f, w=4000, h=1200);
t = tiledlayout(2, 4, TileIndexing="columnmajor");

for i=1:numel(periods)
    % Set relevant parameters
    set_param(sprintf("%s/Sine Wave", mdl), "Frequency", sprintf("%f", 2*pi*freqs(i)));
    set_param(sprintf("%s/Repeating Sequence", mdl), "rep_seq_t", sprintf("[0 0.5 1 %d]", periods(i)));
    % Run simulation
    outs = sim(mdl);
    % Plot spectral analysis on our figure
    plotSpec(t, outs.simout.time, outs.simout.data, N, sprintf("Period: %d sec, Frequency: %f Hz", periods(i), freqs(i)));
end

% Title toplevel
title(t, "Modulation Analysis", FontSize=ff*1.5);
% Export full graph
e("plots/3_q442.svg")

% Clean up
close_system(mdl, 0);
Simulink.sdi.clear;
```

`q443_1.m`

```
% Add the simulink files to path
addpath("simu")

% The model we are working on
mdl = 'lab3CTSys1';

% Load simulink model and turn on logging
system = load_system(mdl);
set_param(mdl, ReturnWorkspaceOutputs="on")

% Our output plot length
N = 256;

% Plot each spectral analysis
f = figure;
[ff, e] = helpers(f, h=1500);
t = tiledlayout(3, 1);

% Run simulation
outs = sim(mdl);
% Plot spectral analysis on our figure
plotNet(t, outs.simout.time, outs.simin.data, outs.simout.data, N)

% Title toplevel
title(t, "Network Analysis", FontSize=ff*1.5);
% Export full graph
e("plots/4_q443.svg")

% Clean up
close_system(mdl, 0);
Simulink.sdi.clear;
```

`q443_2.m`

```
% Add the simulink files to path
addpath("simu")

% The model we are working on
mdl = 'lab3CTSys2';

% Load simulink model and turn on logging
system = load_system(mdl);
set_param(mdl, ReturnWorkspaceOutputs="on")

% Set up plots
f = figure;
[ff, e] = helpers(f);

% Run simulation
outs = sim(mdl);
% Plot impulse response
plot(outs.simout.time, outs.simout.data)
xlabel("Time in seconds")
title("Impulse Response")

% Export graph
e("plots/5_q443.svg")

% Clean up
close_system(mdl, 0);
Simulink.sdi.clear;
```

`q451.m`

```
% Define out DTFT function
DTFT = @(x, n0) @(w) arrayfun(@(o) sum(x .* exp(-1j .* o .* ((1:length(x)) + n0 - 1))), w);

% Define constants
samples = 100;
dw = 0.001;
w = -pi:dw:pi;

% Define functions from lab question
funcs = { ([1]) ([0 0 0 0 0 1]) (0.5.^(1:samples)) };

% Precalculate DTFT functions
outputs = cellfun(@(f) DTFT(f, 0), funcs, UniformOutput=false);

% Set up canvas
f = figure;
[ff, e] = helpers(f, w=2000);
t = tiledlayout(2, 3, TileIndexing="columnmajor");

% Calculate for omegas listed and plot angle and amplitude
for i = 1:numel(outputs)
    o = outputs{i}(w);
    nexttile
    plot(w, abs(o))
    xlabel("Omega")
    ylabel("Amplitude")
    ylim([0 1.5])
    title(sprintf('Function %d', i))
    nexttile
    plot(w, angle(o))
    xlabel("Omega")
    ylabel("Phase")
end

% Title toplevel
title(t, "DTFT Analysis", FontSize=ff*1.5);
% Export graphic
e('plots/6_q451.svg')
```

`q452.m`

```
% Add the simulink files to path
addpath("simu")

% The model we are working on
mdl = 'lab3DTSys';

% Load simulink model and turn on logging
system = load_system(mdl);
set_param(mdl, ReturnWorkspaceOutputs="on")

titles = { "Sin frequency=pi/16", "Sin frequency=pi/8", "Sin frequency=pi/4", "Impulse" };

% Our output plot length
N = 100;
ran = 1:N;

% Plot each spectral analysis
f = figure;
[ff, e] = helpers(f, w=2500, h=1200);
t = tiledlayout(2, 2);

for i=1:numel(titles)
    % Set relevant parameters
    set_param(sprintf("%s/Constant", mdl), "Value", sprintf("%d", i));
    % Run simulation
    outs = sim(mdl);

    % Find all maximums and display
    maxes = islocalmax(outs.simout.data);
    maxvals = outs.simout.data(maxes);
    fprintf("%s max: %f\n", titles{i}, maxvals(numel(maxvals)))

    % Plot spectral analysis on our figure
    nexttile
    hold on
    stairs(outs.simout.time(ran), outs.simout.data(ran));
    stairs(outs.simin.time(ran), outs.simin.data(ran));
    xlabel("Time");
    title(titles{i})
end

% Title toplevel
title(t, "Discrete Time System Analysis", FontSize=ff*1.5);
% Export full graph
e("plots/7_q452.svg")

% Clean up
close_system(mdl, 0);
Simulink.sdi.clear;
```

`runall.m`

```
% Clean workspace
close all
clear all

warning('off', 'Simulink:SFunctions:ObsoleteMFileSFcn');

% Define working functions
functions = { @q422, @q441, @q442, @q443_1, @q443_2, @q451, @q452 };

% Clean after each function
for i=1:numel(functions)
    functions{i}();
    clearvars -except functions i
end

close all
clear all
```